Projekt semestralny – skaner amplifikacji

KACPER SMYKLA, MICHAŁ JOŃSKI

HTTPS://GITHUB.COM/KACPER79/DDOS-TOOLS

Cel projektu

Celem projektu jest stworzenie skanera adresów IP, który sprawdza czy dana ofiara jest podatna na ataki amplifikacyjne

Zaimplementowano 3 protokoły w skanerze: DNS (wraz z rekordami EDNS0), NTP (read-values), SNMP

W przypadku otrzymania pakietu z powrotem obliczany jest współczynnik amplifikacji

Projekt został zaimplementowany w C++ I nie wymaga żadnych bibliotek

Projekt obecnie działa tylko na Linuksie, gdyż API Sockets nie jest identyczne na Windows

Przypadki użycia programu

ddos-tools

Collection of simple tools meant for scanning insecure network configurations.

NTP

Sends NTP Read-value request to selected NTP server

Usage:
`./ddos n [ip_adress]`

SNMP

Program requests common `1.3.6.1.2.1.1.5.0` oid, If it gets a response, It starts scanning process, requesting each possible oid constrainted by arguments provided by user.

Usage:
`./ddos s [ip_adress] [oid_length] [oid_max_number]`

DNS

Sends DNS Request packet to selected IP DNS Resolver to check up on a domain specified by user.

Usage:
`./ddos d [ip_adress] [domain_name]`

Requirements

- Linux
- net-snmp
- CMake

Przykład użycia protokołu DNS

```
[noanime@SOLF build]$ ./AmplificationScanner d 1.1.1.1 isc.org
DNS Request (ANY) sent to 1.1.1.1
Received some response
Amplification Factor: 5.44444
Response data:
RESPONSE ID: 4660 | Op: 0 | Flags 33152 | Records: 2 Ans, 1 Add, 0 Auth, 1 Ques
Answers:
  [Name: isc.org, Type: 48, DataLen: 68]
  [Name: isc.org, Type: 48, DataLen: 68]
Additionals:
  [Name: , Type: 41, DataLen: 0]
```

Przykład użycia protokołu NTP

```
[noanime@SOLF build]$ ./AmplificationScanner n 212.110.131.199
The request was sent
Received some response
Amplification Factor: 26.6667
Response data: version="4", processor="unknown", system="UNIX", leap=0, stratum=4,
precision=-28, rootdelay=90.683, rootdispersion=73.810, peer=657,
refid=10.0.8.50, reftime=0xED1F8430.9D4AA183, poll=6,
clock=0xED1F8604.230F3378, state=4, offset=20.701, frequency=18.215,
jitter=4.334, noise=0.564, stability=0.082
```

Przykład użycia protokołu SNMP

SNMP: start

Timeout: No Response from 192.168.100.1.

192.168.100.1:

Timeout: No Response from 192.168.100.20.

192.168.100.20:

Timeout: No Response from 192.168.100.29.

192.168.100.29:

Timeout: No Response from 192.168.100.55.

192.168.100.55:

Timeout: No Response from 192.168.100.145.

192.168.100.145:

Timeout: No Response from 192.168.100.221.

192.168.100.221:

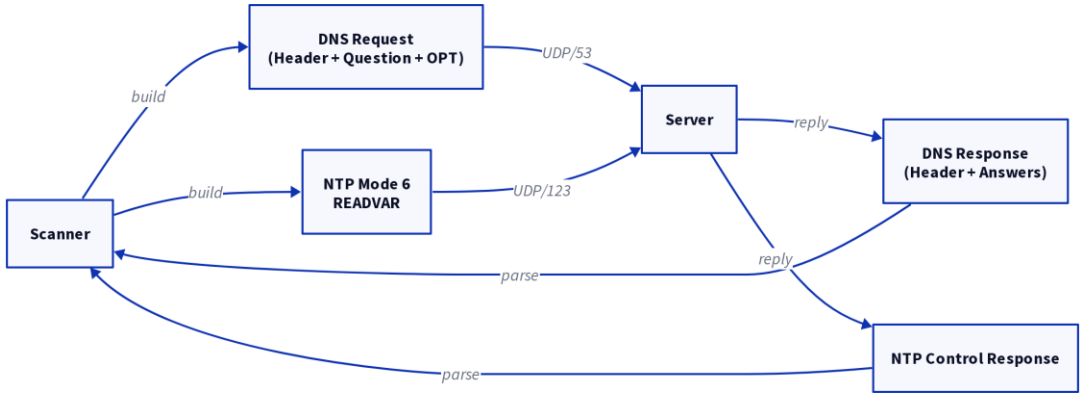
192.168.100.254: SNMPv2-MIB::sysDescr.0 = STRING: CSS106-1G-4P-1S SwOS v2.11

Timeout: No Response from 192.168.100.140.

192.168.100.140: SNMP: finish

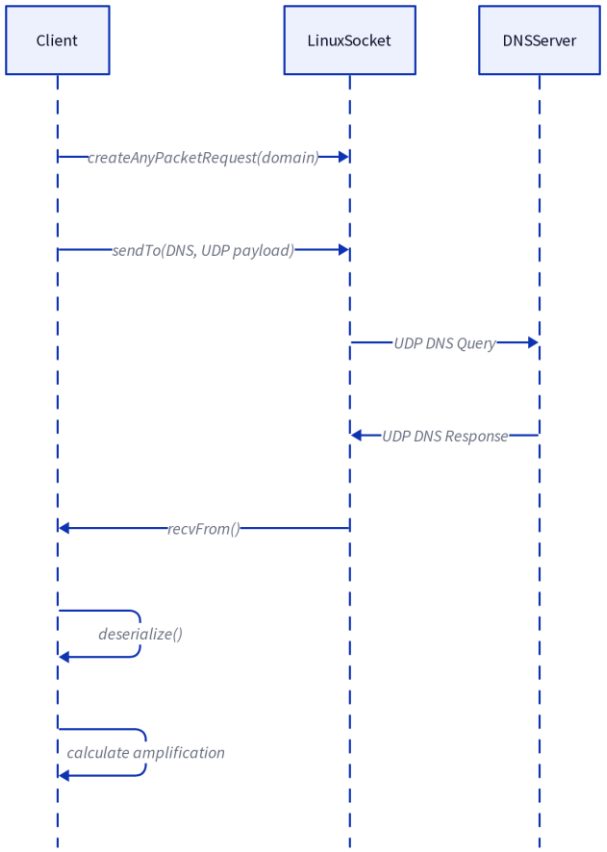
192.168.100.1

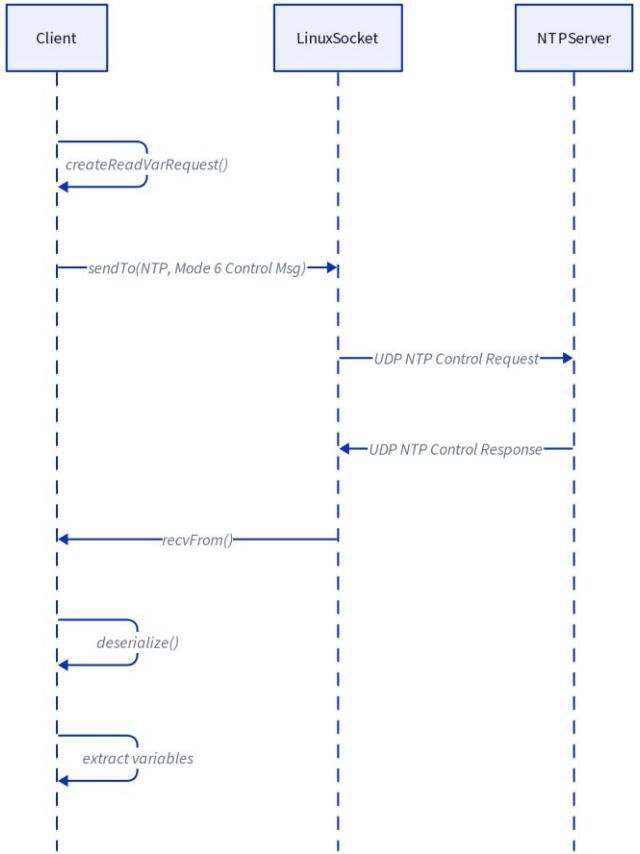
Diagramy #1

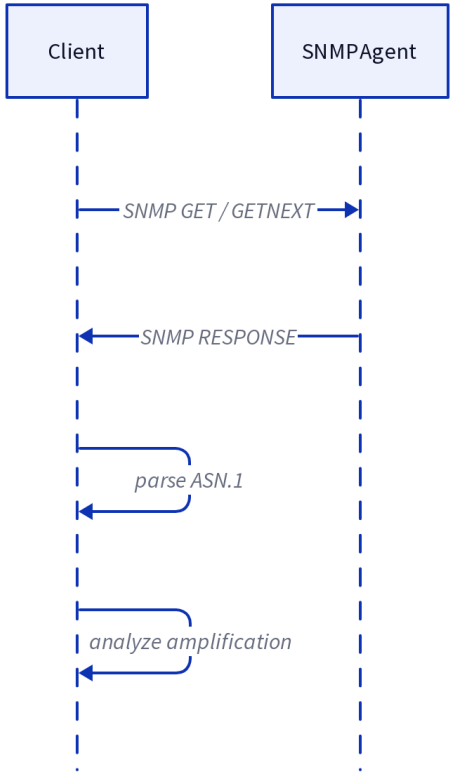

DNS Request
(Header + Question + OPT)
UDP/53
build
Server
reply
DNS Response
(Header + Answers)
build
NTP Mode 6
READVAR
UDP/123
Scanner
parse
reply
NTP Control Response
parse

LinuxSocket
- sockfd int
- isIPv6 bool
+ LinuxSocket(endpoint Endpoint, nonblocking bool) void
+ ~LinuxSocket() void
implements
ISocket
+ open() bool
+ close() void
+ bind(bindTo Endpoint) void
+ setNonBlocking(enabled bool) void
+ nativeHandle() int
Endpoint
+ address char[22]
+ port u16
DnsHeader
+ id u16
+ flags u16
+ qdcount u16
+ ancount u16
+ nscount u16
+ arcount u16
has
DNSPacket
+ header DnsHeader
+ questions vector<DnsQuestion>
+ answers vector<DnsResourceRecord>
+ authorities vector<DnsResourceRecord>
+ additionals vector<DnsResourceRecord>
+ serialize() vector<u8>
+ deserialize(buffer vector<u8>) DNSPacket
+ createAnyPacketRequest(domain string) DNSPacket
+ printAsMessage() string
contains
DnsQuestion
+ qname string
+ qtype u16
+ qclass u16
contains
DnsResourceRecord
+ name string
+ type u16
+ rrclass u16
+ ttl u32
+ rdata vector<u8>
NTPControlPacket
+ li_vn_mode u8
+ r_e_m_op u8
+ sequence u16
+ status u16
+ assoc_id u16
+ offset u16
+ count u16
+ data vector<u8>
+ serialize() vector<u8>
+ deserialize(buffer vector<u8>) NTPControlPacket
+ createReadVarRequest() NTPControlPacket

Diagramy #2 - Sekwencji







Kod: Struktury danych na pakiety

```cpp
// Struct for NTPv2 control (Mode 6) request/response
// https://docs.ntpsec.org/latest/mode6.html
// Op = READVAR (pobranie zmiennych systemowych)
struct NTPControlPacket {
    // Header (12B)
    u8 li_vn_mode;      // Leap Indicator (2), Version (3), Mode (3)
    u8 r_e_m_op;        // Response (1), Error (1), More (1), OpCode (5)
    u16 sequence;       // Sequence Number
    u16 status;         // Status Word
    u16 assoc_id;       // Association ID
    u16 offset;         // Offset
    u16 count;          // Count (długość danych w bajtach)

    // Optional Payload
    std::vector<u8> data;
```

```cpp
struct DnsHeader{
    u16 id;
    u16 flags;
    u16 qdcount;
    u16 ancount;
    u16 nscount;
    u16 arcount;
};

struct DnsResourceRecord{
    str name;
    u16 type;
    u16 rrclass;
    u32 ttl;
    std::vector<u8> rdata; //response data
};

struct DnsQuestion {
    str qname;     // "www.example.com"
    u16 qtype;        // np. 1 (A), 255 (ANY)
    u16 qclass;       // 1 (IN)
};

struct DNSPacket {
    DnsHeader header;
    std::vector<DnsQuestion> questions;
    std::vector<DnsResourceRecord> answers;
    std::vector<DnsResourceRecord> authorities;
    std::vector<DnsResourceRecord> additionals;
```

```cpp
class ISocket {
public:
    virtual ~ISocket() = default;

    virtual bool open() = 0;
    virtual void close() = 0;
    virtual void bind(const Endpoint& bindTo) = 0;
    virtual void setNonBlocking(bool enabled) = 0;
    virtual bool sendTo(const Endpoint& dst, const u8* data, sz len) = 0;
    virtual int recvFrom(Endpoint& src, u8* buffer, sz len) = 0;
    virtual int nativeHandle() const = 0;
};
```

Kod: Tworzenie pakietów zapytań

```
static NTPControlPacket createReadVarRequest() {
    NTPControlPacket pkt;

    // 1. Version Number (VN) = 2
    // 2. Mode = 6 (Control Message).
    // 00 (LI) 010 (VN=2) 110 (Mode=6) -> 00010110 -> 0x16
    pkt.li_vn_mode = (0 << 6) | (2 << 3) | 6;

    // 3. OpCode = 2 (READ_VARIABLES).
    // 0 (R) 0 (E) 0 (M) 00010 (Op=2) -> 0x02
    pkt.r_e_m_op = 2;

    // 4. The rest of the fields can be zero
    pkt.sequence = htons(1); // Example seq
    pkt.assoc_id = 0;        // 0 - this system and not some chosen peer
    pkt.count = 0;           // No payload data

    return pkt;
}
```

```
//TODO: Implement request
static DNSPacket createAnyPacketRequest(const std::string& domain){
    DNSPacket pkt;
    pkt.header.id = 0x1234; // Losowe ID
    pkt.header.flags = 0x0100; // Standard query, recursion desired
    pkt.header.qdcount = 1;
    pkt.header.ancount = 0;
    pkt.header.nscount = 0;
    pkt.header.arcount = 1;

    DnsQuestion q;
    q.qname = domain;

    //QTYPES CAN BE DIFFERENT, NOBODY RESPONDS TO ANY REQUESTS
    //ANY - 0x00FF, DNSKEY - 0x0030, RRSIG - 0x002B
    q.qtype = 0x0030;
    q.qclass = 0x0001; // Klasa IN (Internet)
    pkt.questions.push_back(q);

    //EDNS handling
    DnsResourceRecord opt;
    opt.name = "";
    opt.type = 41;        // OPT
    opt.rrclass = 4096;   // UDP payload size
    opt.ttl = 0;
    opt.rdata = {};
    pkt.additionals.push_back(opt);

    return pkt;
}
```

Kod: Metody serializujące pakiet na bajty

```cpp
std::vector<u8> serialize() const{
    std::vector<u8> buffer;
    buffer.reserve(12 + questions.size() * 3);

    auto push16 = [&buffer](u16 val) {
        u16 be_val = htons(val);
        const u8* bytes = reinterpret_cast<const u8*>(&be_val);
        buffer.push_back(bytes[0]);
        buffer.push_back(bytes[1]);
    };
    auto push32 = [&buffer](u32 val) {
        u32 be_val = htonl(val);
        const u8* bytes = reinterpret_cast<const u8*>(&be_val);
        buffer.push_back(bytes[0]);
        buffer.push_back(bytes[1]);
        buffer.push_back(bytes[2]);
        buffer.push_back(bytes[3]);
    };

    auto pushString = [&buffer](str val){
        std::stringstream ss(val);
        std::string label;
        while (std::getline(ss, label, '.')) {
            buffer.push_back(static_cast<u8>(label.length()));
            for (char c : label) buffer.push_back(c);
        }
        buffer.push_back(0);
    };

    // Header
    push16(header.id);
    push16(header.flags);
    push16(header.qdcount);
    push16(header.ancount);
    push16(header.nscount);
    push16(header.arcount);
```

```cpp
// Questions
for (auto& q : questions) {
    //encode_qname(q.qname, buffer);
    pushString(q.qname);
    push16(q.qtype);
    push16(q.qclass);
}

// Resource records (answers / ns / additional)
auto encode_rr = [&](const DnsResourceRecord& rr) {
    //encode_qname(rr.name, buffer);
    pushString(rr.name);
    push16(rr.type);
    push16(rr.rrclass);
    push32(rr.ttl);
    push16(rr.rdata.size());
    buffer.insert(buffer.end(), rr.rdata.begin(), rr.rdata.end());
};

for (auto& rr : answers) encode_rr(rr);
for (auto& rr : authorities) encode_rr(rr);
for (auto& rr : additionals) encode_rr(rr);

return buffer;
```

```cpp
std::vector<u8> serialize() const {
    std::vector<u8> buffer;
    buffer.reserve(12 + data.size());

    buffer.push_back(li_vn_mode);
    buffer.push_back(r_e_m_op);

    // Helper lambda to push u16 in Big Endian
    auto push_u16 = [&](u16 val) {
        u16 be_val = htons(val); // Host to Network Short
        const u8* bytes = reinterpret_cast<const u8*>(&be_val);
        buffer.push_back(bytes[0]);
        buffer.push_back(bytes[1]);
    };

    push_u16(sequence);
    push_u16(status);
    push_u16(assoc_id);
    push_u16(offset);
    push_u16(count);

    buffer.insert(buffer.end(), data.begin(), data.end());

    return buffer;
}
```

Kod: Metody deserializujące pakiety

```cpp
//TODO: implement deserialization
static DNSPacket deserialize(const std::vector<u8>& buffer){
    DNSPacket pkt;
    if (buffer.size() < 12) return pkt;

    sz pos = 0;
    auto read16 = [&]() {
        u16 val;
        memcpy(&val, &buffer[pos], 2);
        pos += 2;
        return ntohs(val);
    };
    auto read32 = [&]() {
        u32 val;
        memcpy(&val, &buffer[pos], 4);
        pos += 4;
        return ntohl(val);
    };

    auto decode_qname = [&buffer](sz& current_pos) -> str {
        str result = "";
        sz p = current_pos;
        bool jumped = false;
        sz offset_after_jump = 0;
        int safety_count = 0;

        while (buffer[p] != 0 && safety_count++ < 128) {
            if ((buffer[p] & 0xC0) == 0xC0) { // Kompresja DNS
                u16 offset = ((buffer[p] & 0x3F) << 8) | buffer[p + 1];
                if (!jumped) offset_after_jump = p + 2;
                p = offset;
                jumped = true;
                continue;
            }
            u8 len = buffer[p++];
            if (!result.empty()) result += ".";
            for (u8 i = 0; i < len && p < buffer.size(); i++) {
                result += (char)buffer[p++];
            }
        }
        current_pos = jumped ? offset_after_jump : p + 1;
        return result;
    };

    pkt.header.id = read16();
    pkt.header.flags = read16();
    pkt.header.qdcount = read16();
    pkt.header.ancount = read16();
    pkt.header.nscount = read16();
    pkt.header.arcount = read16();
```

```cpp
auto decode_rr = [&]() -> DnsResourceRecord {
    DnsResourceRecord rr;
    rr.name = decode_qname(pos);
    rr.type = read16();
    rr.rrclass = read16();
    rr.ttl = read32();
    u16 rdlen = read16();
    for (u16 j = 0; j < rdlen; ++j) rr.rdata.push_back(buffer[pos++]);
    return rr;
};

for (int i = 0; i < pkt.header.qdcount; ++i) {
    DnsQuestion question;
    question.qname = decode_qname(pos); // Nazwa
    question.qtype = read16();
    question.qclass = read16();
    pkt.questions.push_back(question);
}

for (int i = 0; i < pkt.header.ancount; ++i) pkt.answers.push_back(decode_rr());
for (int i = 0; i < pkt.header.nscount; ++i) pkt.authorities.push_back(decode_rr());
for (int i = 0; i < pkt.header.arcount; ++i) pkt.additionals.push_back(decode_rr());

return pkt;
```

```cpp
static NTPControlPacket deserialize(const std::vector<u8>& buffer) {
    NTPControlPacket pkt;
    if (buffer.size() < 12) return pkt; //too short

    pkt.li_vn_mode = buffer[0];
    pkt.r_e_m_op = buffer[1];

    auto read_u16 = [&](size_t idx) -> u16 {
        u16 val;
        std::memcpy(&val, &buffer[idx], 2);
        return ntohs(val);
    };

    pkt.sequence = read_u16(2);
    pkt.status   = read_u16(4);
    pkt.assoc_id = read_u16(6);
    pkt.offset   = read_u16(8);
    pkt.count    = read_u16(10);

    // Pobranie danych (jeśli są)
    if (buffer.size() > 12) {
        pkt.data.assign(buffer.begin() + 12, buffer.end());
    }

    return pkt;
}
```